

Shallower ReLU Network Representations via Exact Linear Algebra

Kilian Rueß^{*1}, Gennadiy Averkov², Florestan Brunck³, Moritz Grillo⁴, Christoph Hertrich¹, Georg Loho⁵, Jack Stade³, Moritz Stargalla¹, Matthew Sun⁶, and Martin Winter⁴

¹University of Technology Nuremberg, {kilian.ruess, christoph.hertrich, moritz.stargalla}@utn.de

²Brandenburg University of Technology Cottbus–Senftenberg, averkov@b-tu.de

³University of Copenhagen, {flbr, jast}@di.ku.dk

⁴Max Planck Institute for Mathematics in the Sciences, {moritz.grillo, martin.winter}@mis.mpg.de

⁵Freie Universität Berlin, georg.loho@math.fu-berlin.de

⁶mattsun1@mit.edu

July 27, 2026

Abstract

We prove that the maximum of n real numbers is exactly representable by a ReLU network with two hidden layers for every $n \leq 10$. The constructions are obtained by reducing the problem to exact rational linear algebra: after a symmetry reduction, the necessary cancellations are encoded in finite linear systems over $\mathbb{Q}$, which we solve and verify computationally. The representation of $\max_{10}$ has a structured first hidden layer consisting only of pairwise maxima, a feature that allows it to be recursively substituted into larger networks. We use this to show that for every $n > 10$, the maximum $\max_n$ can be exactly represented with $\lceil \log_5(n/2) \rceil + 1 < \log_5(n) + 1.5694$ hidden layers. Via the generalized hinging-hyperplane representation [14], the same depth bound holds for all continuous piecewise-linear functions on $\mathbb{R}^d$, with $d + 1$ in place of n . In particular, every continuous piecewise-linear function on $\mathbb{R}^d$ for $d \leq 9$ admits a two-hidden-layer ReLU representation. Our results improve on [3, STOC’26]. In that work, the authors established a two-hidden-layer representation for $\max_5$ and an upper bound of $\lceil \log_3(n - 2) \rceil + 1$ hidden layers for $\max_n$.

1 Introduction

A ReLU *network* with D hidden layers is specified by affine transformations $T^{(i)}: \mathbb{R}^{n_{i-1}} \rightarrow \mathbb{R}^{n_i}$, for $i = 1, \dots, D + 1$, and computes

$$f = T^{(D+1)} \circ \text{ReLU} \circ \dots \circ \text{ReLU} \circ T^{(1)},$$

where the activation function $\text{ReLU}(x) := \max\{0, x\}$ is applied coordinate-wise. With this convention, the first D affine maps, together with the following applications of ReLU, form the D *hidden layers*, and $T^{(D+1)}$ is the output map.

Every such network computes a *continuous piecewise-linear function* (CPWL) with finitely many affine pieces, and conversely every CPWL function can be represented by a ReLU network [1]. Exact representation by ReLU networks therefore leads to quantitative questions about CPWL

^{*}Lead author; all other authors ordered alphabetically.

functions: how many neurons are required, and how many hidden layers are required, to represent a given function? This paper concerns the second question when the width is unrestricted. We study it through the maximum function

$$\max_n(x) := \max\{x_1, \dots, x_n\}.$$

The reason for this reduction is the generalized hinging-hyperplane representation of Wang and Sun [14]. Let CPWL_d denote the space of CPWL functions $f: \mathbb{R}^d \rightarrow \mathbb{R}$. Every $f \in \text{CPWL}_d$ can be written as a signed linear combination of maxima of $d + 1$ affine functions: there are signs $\sigma_i \in \{\pm 1\}$ and affine maps $A_i: \mathbb{R}^d \rightarrow \mathbb{R}^{d+1}$, for $i = 1, \dots, s$, such that

$$f = \sum_{i=1}^s \sigma_i \max_{d+1}(A_i(x)). \quad (1)$$

Conversely, a lower bound on the depth for representing $\max_{d+1}$ implies the same lower bound for representing $\max\{0, x_1, \dots, x_d\}$, and therefore for the worst-case depth required to represent arbitrary functions in CPWL_d , compare the discussion in [7]. In that sense, the maximum function can be thought of as being the “hardest” function to represent for neural networks with fixed depth and arbitrary width.

1.1 Known bounds

The starting point is the elementary identity

$$\max\{x_1, x_2\} = x_1 + \text{ReLU}(x_2 - x_1)$$

which gives a one-hidden-layer representation of $\max_2$. Iterating this identity in a balanced binary tree gives $\max_n$ in $\lceil \log_2 n \rceil$ hidden layers, and hence gives every function in CPWL_d in $\lceil \log_2(d + 1) \rceil$ hidden layers [1].

The corresponding lower-bound problem is open in general. Hertrich, Basu, Di Summa, and Skutella [7] formulated the unrestricted-width depth question and connected it with polyhedral and tropical methods. Their work motivated the conjecture that the binary-tree depth is optimal for the maximum function. This conjectured $\lceil \log_2 n \rceil$ lower bound was proved by Haase, Hertrich, and Loho [6] for integral-weight ReLU networks. For networks with decimal-fraction weights Averkov, Hojny, and Merkert [2] proved the weaker lower bound $\lceil \log_3 n \rceil$. Further lower bounds are known under braid-fan compatibility assumptions [5] and under width restrictions at fixed depth [12].

For unrestricted real weights, however, the binary-tree construction is not optimal. Bakaev, Brunck, Hertrich, Stade, and Yehudayoff [3] constructed a two-hidden-layer representation of $\max_5$ and used it to represent $\max_n$, for $n \geq 4$, with $\lceil \log_3(n - 2) \rceil + 1$ hidden layers. Together with the Wang–Sun decomposition, this gives $\lceil \log_3(d - 1) \rceil + 1$ hidden layers for all functions in CPWL_d , for $d \geq 3$. Thus, over the real numbers, the problem is not only to prove lower bounds. One must also determine which higher-arity maximum functions admit shallow representations, and which structural features of such representations allow them to be composed recursively.

Because ReLU is a nonpolynomial activation function, neural networks with a single hidden layer and affine hidden units can *approximate* any continuous function on a compact domain to arbitrary accuracy [9]. The problem of approximating $\max_n$ using ReLU neural networks has been studied previously [13]. Here, in contrast, we focus on the exact representation of piecewise-linear functions by ReLU neural networks over the entire space $\mathbb{R}^d$.

1.2 Our results

We record the three main representation results of this paper below. They will be restated in their appropriate settings and proved in the subsequent sections.

Theorem 1.1. *For every $n \leq 10$, the function*

$$\max_n(x_1, \dots, x_n) = \max\{x_1, \dots, x_n\}$$

admits an exact ReLU representation with two hidden layers. This representation can be chosen such that the first hidden layer only consists of groups of neurons, each group collectively computing a pairwise maximum of coordinates, that is, $\max\{x_i, x_j\}$.

The structured first layer in the $\max_n$ representation is preserved by the recursive substitution introduced in Section 5. More precisely, a first-layer comparison between two intermediate maxima can be replaced by a copy of the $\max_{10}$ block. Each such substitution increases the number of hidden layers by one while multiplying the number of input variables represented by five. This yields the following general bound.

Corollary 1.2. *For every integer $n \geq 3$, the function $\max_n$ has an exact ReLU representation whose number of hidden layers is bounded by*

$$2 + \max\left\{0, \left\lceil \log_5 \left(\frac{n}{10} \right) \right\rceil \right\} = \left\lceil \log_5 \left(\frac{n}{2} \right) \right\rceil + 1.$$

Combining this construction with (1) gives the corresponding result for arbitrary continuous piecewise-linear functions.

Corollary 1.3. *For $d \geq 2$, every function in CPWL_d is exactly representable by a ReLU network with at most*

$$\left\lceil \log_5 \left(\frac{d+1}{2} \right) \right\rceil + 1$$

hidden layers. In particular, every function in CPWL_d with $d \leq 9$ admits an exact representation with two hidden layers.

1.3 Idea of the construction

In describing the construction, it is convenient to allow binary maximum gates with affine inputs. Such gates are also known as rank-2 maxout units [4] and have been used in previous work; see, for example, [1, 5]. This does not change the number of hidden layers, since

$$\max\{L_1(x), L_2(x)\} = \text{ReLU}(L_1(x)) - \text{ReLU}(-L_1(x)) + \text{ReLU}(L_2(x) - L_1(x)),$$

at the cost of increasing the width of the network by at most a constant factor of 3. Thus, for the depth bounds considered here, this max-gate model is equivalent to the usual ReLU model.

Even though it is not necessary for the formal correctness of our construction, for understanding the intuition and some theoretical statements guiding our practical computations, it is useful to take a geometric perspective that relates CPWL functions to polytopes. This perspective has been used in several previous works on this problem, see [7, 6, 2, 3]. A *polytope* is the convex hull of finitely many points. For a convex, positively homogeneous CPWL function $x \mapsto \max_{i=1}^n \{a_i^\top x\}$, the corresponding *Newton polytope* is defined as $\text{conv}\{a_i : i = 1, \dots, n\}$. Conversely, for a given polytope $P \subset \mathbb{R}^n$, its support function is

$$h_P(u) = \max_{p \in P} \langle u, p \rangle.$$

Taking the Newton polytope and taking support functions are inverse operations of each other, inducing a duality between CPWL functions and polytopes. This duality is compatible with several operations, namely, function addition corresponds to Minkowski addition of polytopes, taking maxima corresponds to convex hulls, and multiplication with a positive scalar corresponds to dilation of polytopes; effectively turning the duality into an isomorphism with respect to those operations. This correspondence now allows us to freely switch between the perspectives of CPWL functions and polytopes, thereby enabling proofs about neural network expressivity using polyhedral geometry.

The target $\max_n$ is the support function of the simplex $\text{conv}\{e_1, \dots, e_n\}$, while each pairwise maximum $\max\{x_i, x_j\}$ is the support function of the segment $[e_i, e_j]$. Thus, sums of pairwise maxima and maxima of such sums correspond, respectively, to Minkowski sums and convex hulls. This motivates our ansatz in terms of support functions of explicit polytopes.

Based on the S_n -symmetry of the simplex, one can symmetrize any solution, that is, combine all solutions in the orbit of permuting variables. Therefore, it suffices to establish the resulting identities on the sorted cone

$$\mathcal{C} = \{x \in \mathbb{R}^n \mid x_1 \leq \dots \leq x_n\}.$$

There, each pairwise maximum reduces to a coordinate projection, and the search becomes a finite linear-algebra problem over $\mathbb{Q}$: find a signed combination of the symmetrized functions equal to $x_n = \max_n(x)$. The resulting rational identities are verified exactly and extend to all of $\mathbb{R}^n$ by symmetry.

1.4 Organization

Section 2 explains the support-function linear algebra used to certify identities. Section 3 proves the reduction from symmetric identities to the sorted cone. Section 4 defines the finite ansatz, gives the exact verification method, and states the two-hidden-layer result for $n \leq 10$. Section 5 proves the recursive substitution theorem and derives the base-five depth bound.

2 Signed Minkowski identities as linear algebra

Even though our construction developed later in the paper does not explicitly use the perspective of Newton polytopes, our approach is nevertheless inspired by it. The purpose of this section is to explain this geometric intuition and how it relates to previous work on the problem.

In [3], the authors construct neural networks computing the maximum function by showing that a subdivision of the simplex into structured cells makes it possible to express the simplex as a weighted Minkowski difference of polytopes whose support functions are representable with two hidden layers. This makes it possible to conclude that the maximum function itself can be expressed with two hidden layers.

We follow this idea, but avoid constructing a subdivision explicitly and instead work directly with the weighted Minkowski difference. The basic problem is the following. Given a finite set of promising polytopes $Q_1, \dots, Q_r$, for which we already know that they are representable with two hidden layers, we want to find coefficients $c_i \in \mathbb{R}$ such that the formal Minkowski identity

$$P = \sum_{i=1}^r c_i Q_i$$

holds, allowing the conclusion that P is also representable with two hidden layers. Note that, in this equation, some of coefficients c_i can be negative. By definition, such a weighted Minkowski

sum with potentially negative weights, means the following equality of ordinary Minkowski sums:

$$P + \sum_{c_i < 0} (-c_i)Q_i = \sum_{c_i > 0} c_i Q_i.$$

Passing to support functions translates this into the identity of functions $\mathbb{R}^d \rightarrow \mathbb{R}$

$$h_P = \sum_{i=1}^r c_i h_{Q_i}.$$

Since all support functions involved are piecewise linear, this identity can be reduced to a finite linear system by evaluating it on a suitable finite set of points. This reduction is justified by the following proposition. A *complete polyhedral fan* in $\mathbb{R}^n$ is a finite collection of polyhedral cones with apex at the origin, closed under taking faces, such that the intersection of any two cones is a face of each, and whose union is all of $\mathbb{R}^n$.

Proposition 2.1. *Let Σ be a complete polyhedral fan and consider polytopes $P, Q_1, \dots, Q_r$ such that each of the support functions $h_P, h_{Q_1}, \dots, h_{Q_r}$ is linear on every individual cone of Σ . For each cone $\tau \in \Sigma$, let U_τ be a set of generators of its extreme rays. Let*

$$U = \bigcup_{\tau \in \Sigma} U_\tau.$$

Define

$$A_{ui} = h_{Q_i}(u), \quad b_u = h_P(u) \quad (u \in U).$$

Then

$$h_P = \sum_{i=1}^r c_i h_{Q_i} \iff Ac = b.$$

Proof. The function $h_P - \sum_i c_i h_{Q_i}$ is linear on every cone of Σ . A linear function on a cone vanishes identically if and only if it vanishes on the extreme rays of that cone. Since Σ covers $\mathbb{R}^d$, the global identity is therefore equivalent to the finite system $Ac = b$. $\square$

Proposition 2.1 suggests the following procedure to find a neural network representation of $\max_n$. Let P be the standard simplex $\text{conv}\{e_1, \dots, e_n\}$, which is the Newton polytope of $\max_n$. Select promising candidate polytopes $Q_1, \dots, Q_r$, for which we already know that they admit two-hidden-layer representations. Use Proposition 2.1 to reduce the signed Minkowski identity to a finite linear system and check with exact rational arithmetic if it has a solution. If we find a solution, this implies a representation of $\max_n$ with two hidden layers. If the system has no solution, this means that the set of candidate polytopes is not sufficient to represent $\max_n$.

Remark 2.2. The subdivision used in the $\max_5$ construction in [3] can also be interpreted in this framework. Given its cells and their nonempty intersections, we take the corresponding polytopes as the candidate summands Q_i . Evaluating their support functions on a common refinement of their normal fans and solving the resulting system $Ac = b$ determines the signed coefficients. In this way, the same Minkowski identity can be recovered directly from the pieces, rather than derived during the construction of the subdivision.

3 Symmetrization

Let the symmetric group S_n act on $\mathbb{R}^n$ by permuting coordinates. Explicitly, for $x \in \mathbb{R}^n$ and $\sigma \in S_n$, define

$$(\sigma x)_i := x_{\sigma^{-1}(i)} \quad \text{for } i \in \{1, \dots, n\}.$$

For any function $f: \mathbb{R}^n \rightarrow \mathbb{R}$, define its *symmetrization* by

$$f^{\text{sym}}(x) = \frac{1}{n!} \sum_{\sigma \in S_n} f(\sigma x).$$

We call a function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ *symmetric* if $f^{\text{sym}} = f$.

Proposition 3.1. *Let $g: \mathbb{R}^n \rightarrow \mathbb{R}$ be symmetric. If $g = \sum_{i=1}^n c_i f_i$, then*

$$g = \sum_{i=1}^n c_i f_i^{\text{sym}}.$$

Moreover, if $f, h: \mathbb{R}^n \rightarrow \mathbb{R}$ are symmetric, then $f = h$ on $\mathbb{R}^n$ if and only if $f = h$ on

$$\mathcal{C} = \{x \in \mathbb{R}^n : x_1 \leq x_2 \leq \dots \leq x_n\}.$$

Proof. Symmetrization is linear and fixes symmetric functions, giving the first claim. For the second, every point of $\mathbb{R}^n$ is carried into $\mathcal{C}$ by a coordinate permutation. $\square$

Thus, in order to check whether a symmetric candidate equals $\max_n$, it is sufficient to check whether it equals x_n on $\mathcal{C}$. Note that $\mathcal{C}$ is one cell of the braid arrangement, see [5].

4 Exact search for two-hidden-layer identities

This section describes the finite search space used to find the two-hidden-layer representations. The search has three steps. First we choose a restricted family of two-hidden-layer ReLU blocks whose first hidden layer consists only of pairwise maxima. Then we average these blocks over all coordinate permutations, so that it is enough to test them on the sorted cone. Finally we expand the resulting functions into linear terms and hinge terms and solve the resulting rational linear system.

4.1 The two-layer ansatz

Let

$$\mathcal{E}_n := \{(i, j) \mid 1 \leq i \leq j \leq n\}$$

denote the set of unordered pairs of indices, represented by listing the smaller index first. By definition, diagonal elements (i, i) are included. For $(i, j) \in \mathcal{E}_n$ define

$$m_{ij}(x) = \max\{x_i, x_j\}.$$

Fix $k \in \mathbb{N}$, and let $\mathcal{M}_{n,k}$ denote the set of multisets of cardinality k with elements in $\mathcal{E}_n$. Thus, an element $A \in \mathcal{M}_{n,k}$ is a collection of k pairs from $\mathcal{E}_n$, counted with multiplicity, so the same pair may occur several times. Since $\mathcal{E}_n$ is finite, the set $\mathcal{M}_{n,k}$, and hence the set of pairs

$$\mathcal{M}_{n,k} \times \mathcal{M}_{n,k}$$

is finite. For each $(A, B) \in \mathcal{M}_{n,k} \times \mathcal{M}_{n,k}$, define

$$\Phi_{A,B}(x) = \max \left\{ \sum_{(i,j) \in A} m_{ij}(x), \sum_{(i,j) \in B} m_{ij}(x) \right\},$$

where each sum is taken with multiplicity. This is representable with a two-hidden-layer network: the first hidden layer computes the pairwise maxima m_{ij} , and the second hidden layer computes the maximum of the two displayed linear combinations.

We ask whether $\max_n$ lies in the linear span of all symmetrized functions

$$F_{A,B}(x) = n! \Phi_{A,B}^{\text{sym}}(x) = \sum_{\sigma \in S_n} \Phi_{A,B}(\sigma x), \quad (A, B) \in \mathcal{M}_{n,k} \times \mathcal{M}_{n,k}.$$

The factor $1/n!$ appearing in the corresponding group average is omitted, since it merely rescales the solution coefficients.

For computational or structural purposes, one may restrict the system to a chosen subfamily of pairs (A, B) ; unless stated otherwise, however, the ansatz includes all pairs in

$$\mathcal{M}_{n,k} \times \mathcal{M}_{n,k}.$$

By Proposition 3.1, it suffices to work on the sorted cone

$$\mathcal{C} = \{x \in \mathbb{R}^n \mid x_1 \leq x_2 \leq \dots \leq x_n\}.$$

On $\mathcal{C}$, the first-layer functions m_{ij} collapse to coordinate projections onto the variable with the larger index: namely,

$$m_{ij}(\sigma x) = m_{ij}(x_{\sigma^{-1}(1)}, \dots, x_{\sigma^{-1}(n)}) = x_{\max\{\sigma^{-1}(i), \sigma^{-1}(j)\}}.$$

Thus, after restriction to $\mathcal{C}$, the first layer is linear and introduces no additional breakpoints. For a multiset $A \in \mathcal{M}_{n,k}$ and $\sigma \in S_n$, put

$$\ell_{\sigma,A}(x) := \sum_{(i,j) \in A} x_{\max\{\sigma^{-1}(i), \sigma^{-1}(j)\}}.$$

Then, on $\mathcal{C}$,

$$F_{A,B}(x) = \sum_{\sigma \in S_n} \max\{\ell_{\sigma,A}(x), \ell_{\sigma,B}(x)\}.$$

Using

$$\max\{u, v\} = u + \text{ReLU}(v - u)$$

and collecting identical difference vectors, we obtain

$$F_{A,B}(x) = L_{A,B}(x) + \sum_{d \in D_{A,B}} c_{A,B,d} \text{ReLU}(d^\top x),$$

where $L_{A,B}$ is linear, $D_{A,B} \subset \mathbb{Z}^n$ is finite, and $c_{A,B,d} \in \mathbb{Z}$. Consequently, a representation $\max_n = \sum_{A,B} \lambda_{A,B} F_{A,B}$ is certified by the exact equations

$$\sum_{A,B} \lambda_{A,B} c_{A,B,d} = 0 \quad \text{for all } d \in \bigcup_{A,B} D_{A,B} \tag{2}$$

$$\sum_{A,B} \lambda_{A,B} L_{A,B}(x) = x_n. \tag{3}$$

The first set of equations (2) says that every nonlinear hinge term cancels. The second (3) says that the remaining linear function is x_n , which equals $\max_n$ on the sorted cone $\mathcal{C}$.

4.1.1 Enumeration modulo permutations

The function $F_{A,B}$ is invariant under interchanging A and B , as well as under simultaneously relabelling all indices in both multisets. Hence it suffices to consider pairs (A, B) modulo the equivalence relation

$$(A, B) \sim (B, A) \sim (\tau A, \tau B) \sim (\tau B, \tau A), \quad \tau \in S_n.$$

We use $\{\{\dots\}\}$ to denote multisets, so repetitions indicate multiplicity. Here,

$$\tau A := \{(\min\{\tau(i), \tau(j)\}, \max\{\tau(i), \tau(j)\}) : (i, j) \in A\}$$

with multiplicities retained. We refer to an equivalence class under this relation as a *template*.

Equivalently, a pair (A, B) may be regarded as a two-edge-colored multigraph on the vertex set $\{1, \dots, n\}$. The pairs in A form the edges of one color, those in B form the edges of the other color, and all edge multiplicities are retained. If diagonal pairs are permitted in the ansatz, they are represented by loops. Interchanging A and B corresponds to exchanging the two edge colors, while simultaneous permutation of the indices corresponds to relabelling the vertices. Template enumeration is therefore a colored-multigraph isomorphism problem, with the two colors themselves regarded as interchangeable.

In the implementation, each isomorphism class is replaced by a canonical labelled representative. All certificate coefficients are then aggregated according to this canonical unordered pair before the finite linear system is assembled. The required isomorphism classes can be enumerated efficiently using standard graph-isomorphism software, such as *nauty* [10].

Remark 4.1. Several elementary linear dependencies can be eliminated to reduce the number of constraints in the resulting linear system. For instance, suppose that $d^\top x$ has a fixed sign on $\mathcal{C}$. If $d^\top x \leq 0$ throughout $\mathcal{C}$, then

$$\text{ReLU}(d^\top x) = 0$$

on $\mathcal{C}$. If instead $d^\top x \geq 0$ throughout $\mathcal{C}$, then

$$\text{ReLU}(d^\top x) = d^\top x$$

on $\mathcal{C}$. Hence, in either case, the corresponding hinge direction may be omitted. Moreover, the hinge directions d and $-d$ can be identified using the identity

$$\text{ReLU}(-d^\top x) = \text{ReLU}(d^\top x) - d^\top x.$$

Finally, by positive homogeneity,

$$\text{ReLU}(q \cdot d^\top x) = q \cdot \text{ReLU}(d^\top x), \quad q > 0.$$

Thus, hinge directions that differ by a positive scalar multiple may also be identified.

4.2 The smallest useful value of k

The parameter k controls the expressive power and the size of the computational ansatz. Each argument of $\Phi_{A,B}$ does contain the sum of k pairwise maxima, so increasing k enlarges the class of functions that the ansatz can represent. At the same time, the number of pairs of multisets (A, B) grows rapidly with k . It is therefore natural to ask for the smallest value of k that is not ruled out by theoretical obstructions.

Proposition 4.2. *Let $\Delta_k = \text{conv}\{0, e_1, \dots, e_k\}$. There do not exist polytopes $P_1, \dots, P_m$ with $\dim P_r < k$ and coefficients $c_r \in \mathbb{R}$ such that*

$$h_{\Delta_k} = \sum_{r=1}^m c_r h_{P_r}.$$

Equivalently, a k -simplex is not a signed Minkowski combination of lower-dimensional polytopes.

Proof. This is precisely [8, Corollary 5.2], translated into support-function language. In the terminology of that paper, the assertion is that a simplex is not a zero summand. $\square$

Corollary 4.3. *The exact systems assembled from the ansatz above are solvable only if*

$$k \geq k_{\min} := \left\lfloor \frac{n-1}{2} \right\rfloor.$$

Thus, $k_{\min}$ is the smallest value of k not ruled out by Proposition 4.2.

Proof. For each (i, j) , $m_{ij} = h_{[e_i, e_j]}$. Hence

$$\sum_{(i,j) \in A} m_{ij} = h_{Z_A}, \quad Z_A := \sum_{(i,j) \in A} [e_i, e_j],$$

where $\dim Z_A \leq k$. Therefore

$$\Phi_{A,B} = \max\{h_{Z_A}, h_{Z_B}\} = h_{\text{conv}(Z_A \cup Z_B)}.$$

The convex hull of two polytopes of dimensions at most k has dimension at most $2k+1$. Expanding each symmetrized block into its summands, any solution of the exact system would therefore express $\max_n = h_{\text{conv}\{e_1, \dots, e_n\}}$, the support function of an $(n-1)$ -simplex, as a signed Minkowski combination of polytopes of dimension at most $2k+1$. If $2k+1 < n-1$, this contradicts Proposition 4.2. Thus one must have $2k+1 \geq n-1$, equivalently $k \geq \lfloor (n-1)/2 \rfloor$. $\square$

4.3 Computational results

Theorem 1.1. *For every $n \leq 10$, the function*

$$\max_n(x_1, \dots, x_n) = \max\{x_1, \dots, x_n\}$$

admits an exact ReLU representation with two hidden layers. This representation can be chosen such that the first hidden layer only consists of groups of neurons, each group collectively computing a pairwise maximum of coordinates, that is, $\max\{x_i, x_j\}$.

Proof. The cases $n \leq 4$ are immediate. For each $5 \leq n \leq 10$, let

$$k = \lfloor (n-1)/2 \rfloor.$$

As described in Section 4.1, we form the linear system indexed by all pairs

$$(A, B) \in \mathcal{M}_{n,k} \times \mathcal{M}_{n,k}.$$

We then apply the reduction from Remark 4.1. We solve this system exactly over $\mathbb{Q}$ using an optimized implementation. The resulting solution provides rational coefficients $\lambda_{A,B}$ such that all hinge coefficients vanish and the remaining linear part equals x_n on the sorted cone. By Proposition 3.1, this implies equality on all of $\mathbb{R}^n$ as both functions are symmetric.

Since each block $F_{A,B}$ is constructed from pairwise maxima in the first hidden layer and one further maximum in the second hidden layer, this identity gives a two-hidden-layer ReLU representation.

The resulting certificates can be verified using the conceptually simpler Python verifier provided in the accompanying GitHub repository. This verifier performs all computations using exact rational arithmetic. Although it is substantially slower than the optimized implementation used to obtain the certificates, it provides a transparent and directly reproducible check of the identities. $\square$

As a concrete example, take $n = 6$ and $k = 2$. Let

i	A_i	B_i
1	$\{(1, 2), (1, 2)\}$	$\{(3, 4), (3, 4)\}$
2	$\{(1, 2), (1, 3)\}$	$\{(1, 4), (5, 6)\}$
3	$\{(1, 2), (1, 3)\}$	$\{(4, 5), (4, 6)\}$
4	$\{(1, 2), (3, 4)\}$	$\{(1, 3), (5, 6)\}$

Then one certificate is

$$\max\{x_1, \dots, x_6\} = \frac{1}{720}F_{A_1, B_1} + \frac{1}{360}F_{A_2, B_2} - \frac{1}{1440}F_{A_3, B_3} - \frac{1}{360}F_{A_4, B_4}.$$

Here $F_{A,B}$ is the symmetrized block defined above.

The certificates for $n = 7, \dots, 10$ are much larger and are therefore not reproduced in the body of the paper. They are provided with the accompanying online material¹.

Width of the construction. The constructions above are designed to make the search for two-hidden-layer representations computationally tractable, rather than to minimize network width. Suppose that the certificate is supported on s templates. Symmetrizing each template produces at most $n!$ labelled blocks, so the total number Q of rank-2 maxout neurons satisfies

$$Q \leq sn!.$$

By sharing the required pairwise maxima, the rank-2 maxout realization has first-layer width at most

$$\binom{n+1}{2} = \mathcal{O}(n^2)$$

and second-layer width at most $Q \leq sn!$. The construction can also be converted directly into a standard ReLU network without skip connections. This yields first-layer width at most

$$\binom{n}{2} + 2n = \mathcal{O}(n^2)$$

and second-layer width at most $3Q \leq 3sn!$. Thus the direct symmetrized realization has quadratic width in the first hidden layer and, in the worst case, factorial width in the second. We do not attempt to optimize this crude upper bound, which remains far above the quadratic lower bound for two-hidden-layer representations established in [12].

Remark 4.4. It is known [11, Proposition 2.2] that for $n \geq 3$, one hidden layer is not sufficient to represent $\max_n$. Therefore, together with Theorem 1.1, we obtain that the minimum number of hidden layers to represent $\max_n$ for $n \in \{3, \dots, 10\}$ is exactly two.

¹<https://github.com/kilianar/max-relu-certificates>

5 Recursive composition and depth bounds

There is a standard way to bootstrap any exact network representation of $\max_l$ without imposing assumptions on its internal structure. Given $n > l$ inputs, partition them into groups of size at most l , apply the network to all groups in parallel, and repeat the procedure on the resulting group maxima. If the original network has D hidden layers, this tournament-style construction, together with repeated-coordinate padding for incomplete groups, represents $\max_n$ with $D \lceil \log_l n \rceil$ hidden layers.

The representations constructed in this paper carry additional structure: their first hidden layer consists entirely of pairwise comparisons. Exploiting this structure gives a more efficient bootstrap. Each such comparison can be replaced by a copy of the original two-hidden-layer representation, increasing the number of hidden layers by only one while multiplying the input arity by $\lfloor l/2 \rfloor$. The following theorem formalizes this substitution and its iteration.

Theorem 5.1. *Assume that $\max_l$ has a two-hidden-layer representation $\mathcal{B}_l$ whose first hidden layer consists only of pairwise comparisons $\max\{x_i, x_j\}$, and let $r = \lfloor l/2 \rfloor \geq 2$. Then, for every integer $s \geq 0$, the function $\max_{l^r s}$ has an exact representation with $2 + s$ hidden layers whose first hidden layer again consists only of pairwise comparisons. Consequently, for every $n \geq 1$, the function $\max_n$ has an exact representation with at most*

$$2 + \max\left\{0, \left\lceil \log_r \frac{n}{l} \right\rceil\right\}$$

hidden layers.

Proof. We construct the representations by induction on s , keeping the form of the first hidden layer as an invariant. For $s = 0$, the required representation is $\mathcal{B}_l$.

Suppose that the claim holds for some $s \geq 0$, and set $M = l^r s$. Partition rM inputs into disjoint blocks $S_1, \dots, S_M$, each of size r , and set

$$y_p = \max_{i \in S_p} x_i.$$

Evaluating the representation of $\max_M$ at $(y_1, \dots, y_M)$ gives $\max_p y_p = \max_i x_i$. In this evaluation, each first-layer comparison becomes

$$\max\{y_p, y_q\} = \max_{i \in S_p \cup S_q} x_i,$$

a maximum of $2r \leq l$ inputs. Replace every such comparison by a copy of $\mathcal{B}_l$, using repeated-coordinate padding when $2r < l$ or $p = q$, and absorb the output affine maps of these copies into the affine map preceding the next hidden layer. Thus the first hidden layer is replaced by two hidden layers, giving a representation of $\max_{rM} = \max_{l^{r(s+1)}}$ with $3 + s$ hidden layers. Its first hidden layer consists only of the pairwise comparisons from the copies of $\mathcal{B}_l$, so the invariant is preserved. This is the substitution illustrated in Figure 1 and completes the induction.

Finally, given $n \geq 1$, take

$$s = \max\left\{0, \left\lceil \log_r \frac{n}{l} \right\rceil\right\}.$$

Then $l^{r^s} \geq n$, so repeating one input coordinate as necessary turns the representation of $\max_{l^{r^s}}$ into one of $\max_n$ without changing its depth. $\square$

Corollary 1.2. *For every integer $n \geq 3$, the function $\max_n$ has an exact ReLU representation whose number of hidden layers is bounded by*

$$2 + \max\left\{0, \left\lceil \log_5 \left(\frac{n}{10}\right) \right\rceil\right\} = \left\lceil \log_5 \left(\frac{n}{2}\right) \right\rceil + 1.$$

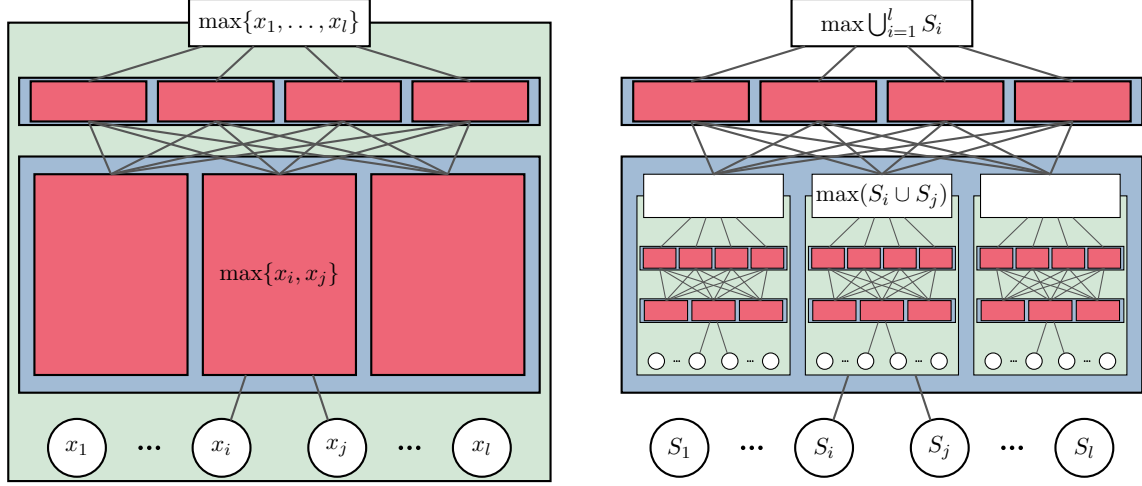


Figure 1: The first bootstrap step. Starting from the two-hidden-layer network $\mathcal{B}_l$ computing $\max_l$, replace each first-layer comparison $\max\{x_i, x_j\}$ by a copy of $\mathcal{B}_l$ computing $\max(S_i \cup S_j)$, where $|S_i| = |S_j| = r = \lfloor l/2 \rfloor$ (using repeated-coordinate padding when $2r < l$). The resulting three-hidden-layer network computes $\max_{lr} = \max_{\lfloor l/2 \rfloor}$.

Proof. Use the two-hidden-layer representation from Theorem 1.1 for $\max_{10}$ and apply Theorem 5.1 with $l = 10$ and $r = 5$. $\square$

Corollary 1.3. *For $d \geq 2$, every function in CPWL_d is exactly representable by a ReLU network with at most*

$$\left\lceil \log_5 \left(\frac{d+1}{2} \right) \right\rceil + 1$$

hidden layers. In particular, every function in CPWL_d with $d \leq 9$ admits an exact representation with two hidden layers.

Proof. By (1), every $f \in \text{CPWL}_d$ is a signed linear combination of maxima of $d+1$ affine forms. Apply Corollary 1.2 to these maxima in parallel and implement their signed linear combination in the final affine layer. This does not increase the number of hidden layers and gives the stated bound. $\square$

Acknowledgments

The authors thank the members of the Neural Polytopes Zulip community and the participants of the Workshop on Polyhedral Geometry for Neural Networks², held in Nuremberg in March 2026, for inspiring exchange.

References

- [1] R. Arora, A. Basu, P. Mianjy, and A. Mukherjee. “Understanding Deep Neural Networks with Rectified Linear Units”. In: International Conference on Learning Representations. 2018.
- [2] G. Averkov, C. Hojny, and M. Merkert. “On the Expressiveness of Rational ReLU Neural Networks With Bounded Depth”. In: The Thirteenth International Conference on Learning Representations. 2025.

²<https://neuralpolytopes.gitlab.io/workshop2026/>

- [3] E. Bakaev, F. Brunck, C. Hertrich, J. Stade, and A. Yehudayoff. “Better Neural Network Expressivity: Subdividing the Simplex”. In: *Proceedings of the 58th Annual ACM Symposium on Theory of Computing*. STOC ’26: 58th Annual ACM Symposium on Theory of Computing. Salt Lake City UT USA: ACM, 2026, pp. 500–507. ISBN: 979-8-4007-2536-4. DOI: 10.1145/3798129.3800768.
- [4] I. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio. “Maxout Networks”. In: *Proceedings of the 30th International Conference on Machine Learning*. International Conference on Machine Learning. PMLR, 2013, pp. 1319–1327.
- [5] M. Grillo, C. Hertrich, and G. Loho. “Depth-Bounds for Neural Networks via the Braid Arrangement”. In: *Advances in Neural Information Processing Systems*. Vol. 38. Curran Associates, Inc., 2025, pp. 127800–127818.
- [6] C. A. Haase, C. Hertrich, and G. Loho. “Lower Bounds on the Depth of Integral ReLU Neural Networks via Lattice Polytopes”. In: *The Eleventh International Conference on Learning Representations*. 2023.
- [7] C. Hertrich, A. Basu, M. Di Summa, and M. Skutella. “Towards Lower Bounds on the Depth of ReLU Neural Networks”. In: *SIAM Journal on Discrete Mathematics* 37.2 (2023), pp. 997–1029. ISSN: 0895-4801, 1095-7146. DOI: 10.1137/22M1489332.
- [8] C. Koutschan, B. Moser, A. Ponomarchuk, and J. Schicho. “Representing Piecewise Linear Functions by Functions with Small Arity”. In: *Applicable Algebra in Engineering, Communication and Computing* 36.4 (2025), pp. 595–610. ISSN: 0938-1279, 1432-0622. DOI: 10.1007/s00200-023-00627-1.
- [9] M. Leshno, V. Y. Lin, A. Pinkus, and S. Schocken. “Multilayer Feedforward Networks with a Nonpolynomial Activation Function Can Approximate Any Function”. In: *Neural Networks* 6.6 (1993), pp. 861–867. ISSN: 08936080. DOI: 10.1016/S0893-6080(05)80131-5.
- [10] B. D. McKay and A. Piperno. “Practical Graph Isomorphism, II”. In: *Journal of Symbolic Computation* 60 (2014), pp. 94–112. ISSN: 07477171. DOI: 10.1016/j.jsc.2013.09.003.
- [11] A. Mukherjee and A. Basu. *Lower Bounds over Boolean Inputs for Deep Neural Networks with ReLU Gates*. TR17-190. Rehovot, Israel, 2017.
- [12] I. Safran. “A Depth Hierarchy for Computing the Maximum in ReLU Networks via Extremal Graph Theory”. In: *Proceedings of Thirty Ninth Conference on Learning Theory*. Vol. 336. PMLR, 2026, pp. 5721–5742.
- [13] I. Safran, D. Reichman, and P. Valiant. “How Many Neurons Does It Take to Approximate the Maximum?” In: *Proceedings of the 2024 ACM-SIAM Symposium on Discrete Algorithms, SODA 2024, Alexandria, VA, USA, January 7-10, 2024*. Ed. by D. P. Woodruff. SIAM, 2024, pp. 3156–3183. DOI: 10.1137/1.9781611977912.113.
- [14] S. Wang and X. Sun. “Generalization of Hinging Hyperplanes”. In: *IEEE Transactions on Information Theory* 51.12 (2005), pp. 4425–4431. ISSN: 0018-9448. DOI: 10.1109/TIT.2005.859246.